Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

Department Of Computer Science & Engineering

Bachelor of Computer Science 6th Semester

System Design

(23CSH-314/23ITH-314)

Unit No. 1 **Chapter No. 1.3.1** **Lecture No.5**

Topic : SOLID Principles

Mayan (E18387) **Technical Trainer(CSE)**

Learning Objectives

1. Explain each SOLID principle with real-time examples.
2. Identify violations of SOLID principles in existing code designs.
3. Apply SOLID principles to design clean low-level components.
4. Analyze design decisions using SOLID guidelines.
5. Relate SOLID principles to system design and scalable architectures.

Topics

1. SOLID Principle
2. Single Responsibility Principle (SRP)
3. Open-Closed Principle (OCP)
4. Liskov Substitution Principle (LSP)
5. Interface Segregation Principle (ISP)
6. Dependency Inversion Principle (DIP)

Introduction to SOLID Principles

SOLID principles are a set of five object-oriented design guidelines that help developers write **maintainable, scalable, and flexible software**.

These principles were introduced by **Robert C. Martin (Uncle Bob)** and are widely used in:

- Enterprise applications
- System design interviews
- Framework and library design

SOLID principles are essential for **clean Low-Level Design (LLD)**.

Overview of SOLID Principles

SOLID stands for :

- **S** – Single Responsibility Principle
- **O** – Open/Closed Principle
- **L** – Liskov Substitution Principle
- **I** – Interface Segregation Principle
- **D** – Dependency Inversion Principle

Each principle addresses a specific design problem in object-oriented systems.

Why SOLID Principles Matter

Without SOLID principles, software systems often become:

- ✓ Difficult to modify
- ✓ Error-prone
- ✓ Hard to test
- ✓ Tightly coupled

SOLID principles help:

- ✓ Reduce code complexity
- ✓ Improve readability
- ✓ Support future changes
- ✓ Align code with real-world requirements

They are heavily used in **microservices, APIs, and scalable systems**.

Single Responsibility Principle (SRP)

Definition:
A class should have **only one reason to change**, meaning it should have **only one responsibility**.

In simple terms:

- Each class should have **only one responsibility**
- Each class should have **only one axis of change**
- Responsibility = "reason to change"

One class → One job
SRP improves clarity and reduces the impact of changes.

SRP – Real-Time Example

SRP Violation - "God Class" Problem

What's Wrong with This Employee Class?

Three Different Responsibilities Mixed Together:

Business Logic Layer
- calculateSalary()
- calculateTax()
- Reason to change: Business rule changes

Data Access Layer
- saveToDatabase()
- loadFromDatabase()
- Reason to change: Database schema/technology changes

Presentation Layer
- printReport()
- generatePDF()
- Reason to change: UI/Report format changes

```java
// ANTI-PATTERN: God Class
class Employee {
    // Business Logic
    public void calculateSalary() { /* ... */ }
    public void calculateTax() { /* ... */ }

    // Data Access
    public void saveToDatabase() { /* ... */ }
    public void loadFromDatabase() { /* ... */ }

    // Presentation
    public void printReport() { /* ... */ }
    public void generatePDF() { /* ... */ }
}
// Three reasons to change: Business rules, Database, Presentation
```

SRP – Refactored Solution:
Four Specialized Classes, Each with ONE Job:

Data Model (POJO)
└─ Employee Class
- Only stores data (id, name, salary)
- Only getters/setters

Responsibility: Represent employee data
Reason to change: Data structure changes

Business Logic
└─ EmployeeCalculator Class
- calculateSalary()
- calculateTax()

Responsibility: Perform calculations
Reason to change: Tax/salary formula changes

Data Access
└─ EmployeeRepository Class
- save(Employee emp)
- load(int id)

Responsibility: Persist/retrieve data
Reason to change: Database technology changes

```java
// Separate responsibilities into different classes

// Business Logic
class EmployeeCalculator {
    public void calculateSalary() { /* ... */ }
    public void calculateTax() { /* ... */ }
}

// Data Access
class EmployeeRepository {
    public void save(Employee emp) { /* ... */ }
    public Employee load(int id) { /* ... */ }
}

// Presentation
class EmployeeReport {
    public void print(Employee emp) { /* ... */ }
    public PDF generatePDF(Employee emp) { /* ... */ }
}

// Data Model (POJO)
class Employee {
    private int id;
    private String name;
    private double salary;
    // Getters and setters only
}
```

Presentation
└─ EmployeeReport Class
- print(Employee emp)
- generatePDF(Employee emp)

Responsibility: Format and display data
Reason to change: Report format/UI changes

Benefits of SRP

- **Easier to understand** - each class does one thing
- **Easier to test** - focused test cases
- **Reduced coupling** - changes affect fewer components
- **Easier maintenance** - isolate changes to specific areas

Open/Closed Principle (OCP)

Definition:
Software entities should be **open for extension but closed for modification**.

This means:
- ❑ New behavior should be added **without changing existing code**
- ❑ OCP reduces the risk of introducing bugs when adding new features.

OCP – Real-Time Example
Violation Example:
A PaymentService with multiple if-else conditions for payment types.

OCP-Compliant Design:
- ❑ Create a Payment interface
- ❑ Implement CardPayment, UPIPayment, WalletPayment
- ❑ Add new payment types without modifying existing code

This approach is common in **payment gateways and plugin systems**.

Violation Example:

```java
class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Circle) {
            Circle c = (Circle) shape;
            return Math.PI * c.radius * c.radius;
        } else if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle) shape;
            return r.width * r.height;
        } else if (shape instanceof Triangle) {
            Triangle t = (Triangle) shape;
            return 0.5 * t.base * t.height;
        }
        throw new IllegalArgumentException("Unknown shape");
        // Adding new shape requires MODIFYING this method
    }
}
```

OCP Implementation Patterns

Solution Using Abstraction:

```java
// Abstract Shape with calculateArea() method
interface Shape {
    double calculateArea();
}

class Circle implements Shape {
    private double radius;
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements Shape {
    private double width, height;
    public double calculateArea() {
        return width * height;
    }
}

// New shapes can be added WITHOUT modifying existing code
class Triangle implements Shape {
    private double base, height;
    public double calculateArea() {
        return 0.5 * base * height;
    }
}

class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.calculateArea();  // No changes needed for new shapes
    }
}
```

Benefits of OCP

- ✓ Safe feature extension
- ✓ Better scalability
- ✓ Cleaner code structure
- ✓ Reduced regression bugs
- ✓ OCP is essential in systems that evolve frequently

Liskov Substitution Principle (LSP)

Definition:

- Objects of a superclass should be replaceable with objects of its subclasses **without affecting correctness**.

In simple terms:

- Child classes should behave like their parent class.

❖ LSP ensures **behavioral consistency**.

Core Concept:

- **"Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."**

Formal Definition:

- If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program.

Violation Example (Classic Rectangle-Square Problem):

```java
class Rectangle {
    protected int width, height;

    public void setWidth(int w) { width = w; }
    public void setHeight(int h) { height = h; }
    public int getArea() { return width * height; }
}

class Square extends Rectangle {
    // Square IS-A Rectangle mathematically, but not behaviorally
    @Override
    public void setWidth(int w) {
        super.setWidth(w);
        super.setHeight(w);  // Violates rectangle behavior!
    }

    @Override
    public void setHeight(int h) {
        super.setHeight(h);
        super.setWidth(h);   // Violates rectangle behavior!
    }
}

// Client code expecting Rectangle behavior breaks!
void testRectangle(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.getArea() == 20;  // FAILS if r is Square!
}
```

Refactored Solution:

```java
// Option 1: Separate inheritance hierarchy
interface Shape {
    int getArea();
}

class Rectangle implements Shape {
    private int width, height;
    // Separate setters maintain rectangle contract
    public int getArea() { return width * height; }
}

class Square implements Shape {
    private int side;
    public int getArea() { return side * side; }
}

// Option 2: Immutable objects
class Rectangle {
    private final int width, height;  // Immutable
    public Rectangle(int w, int h) { width = w; height = h; }
    public int getArea() { return width * height; }
}

class Square {
    private final int side;
    public Square(int s) { side = s; }
    public int getArea() { return side * side; }
}
```

Liskov Substitution Principle (LSP)

Benefits of LSP

1. Reliable polymorphism
2. Predictable behavior
3. Fewer runtime errors
4. LSP is critical when designing **inheritance hierarchies**.

Liskov Substitution Principle (LSP)

Design by Contract:

- **Preconditions** cannot be strengthened in subclass
- **Postconditions** cannot be weakened in subclass
- **Invariants** must be preserved in subclass
- **History constraint** - subclass shouldn't allow state changes parent prohibits

Benefits of LSP

- Reliable polymorphism
- Predictable behavior
- Fewer runtime errors
- LSP is critical when designing **inheritance hierarchies**.

Interface Segregation Principle (ISP)

Definition:
Clients should not be forced to depend on interfaces they do not use.
In simple terms:

- Many small interfaces are better than one large interface.
- ISP avoids unnecessary implementation.

Core Concept:

- "Clients should not be forced to depend on interfaces they do not use."

What It Means:

- Split large interfaces into smaller, specific ones
- Avoid "fat" or "polluted" interfaces
- Prevent classes from implementing irrelevant methods

Violation Example:

```java
// "God Interface" - too many responsibilities
interface Worker {
    void work();
    void eat();
    void sleep();
    void code();
    void test();
    void deploy();
    void maintain();
}

class Programmer implements Worker {
    public void work() { /* ... */ }
    public void eat() { /* ... */ }
    public void sleep() { /* ... */ }
    public void code() { /* ... */ }
    public void test() { /* ... */ }
    public void deploy() { /* ... */ }
    public void maintain() { /* ... */ }
    // Programmer needs all methods - OK
}
```

```java
class Chef implements Worker {
    public void work() { /* ... */ }
    public void eat() { /* ... */ }
    public void sleep() { /* ... */ }
    public void code() { /* ... */ }      // NOT NEEDED!
    public void test() { /* ... */ }      // NOT NEEDED!
    public void deploy() { /* ... */ }    // NOT NEEDED!
    public void maintain() { /* ... */ }  // NOT NEEDED!
    // Chef forced to implement irrelevant methods
}
```

ISP Implementation

```java
// Basic human functions
interface Human {
    void eat();
    void sleep();
}

// Work interface
interface Workable {
    void work();
}

// Programming-specific interface
interface Programmable {
    void code();
    void test();
    void deploy();
    void maintain();
}

// Cooking-specific interface
interface Cookable {
    void prepareMeal();
    void cook();
    void plate();
}
```

```java
// Classes implement only relevant interfaces
class Programmer implements Human, Workable, Programmable {
    public void eat() { /* ... */ }
    public void sleep() { /* ... */ }
    public void work() { /* ... */ }
    public void code() { /* ... */ }
    public void test() { /* ... */ }
    public void deploy() { /* ... */ }
    public void maintain() { /* ... */ }
}

class Chef implements Human, Workable, Cookable {
    public void eat() { /* ... */ }
    public void sleep() { /* ... */ }
    public void work() { /* ... */ }
    public void prepareMeal() { /* ... */ }
    public void cook() { /* ... */ }
    public void plate() { /* ... */ }
}
```

Interface Segregation Principle (ISP)

Benefits of ISP

- Reduced coupling
- Cleaner interfaces
- Easier implementation
- Improved flexibility
- ISP is widely used in **API and SDK design**.

Dependency Inversion Principle (DIP)

Definition:

- High-level modules should not depend on low-level modules.
- Both should depend on **abstractions**.
- Abstractions should not depend on details; details should depend on abstractions.

Core Concept:

- "High-level modules should not depend on low-level modules. Both should depend on abstractions."

Two Key Rules:

1. Abstractions should not depend on details. Details should depend on abstractions.
2. High-level policy should not depend on low-level details.

This approach enables **loose coupling and testability**.

Dependency Inversion Principle (DIP)

Traditional Layered Architecture (Violation):

```
High-level Business Logic
        ↓ depends on
Low-level Data Access
        ↓ depends on
Database Specific Code
```

Inverted Architecture:

```
High-level Business Logic
        ↓ depends on
      Abstract Interface
        ↙ depends on    ↘ depends on
Database Implementation   File Implementation
```

DIP Violation Example

Tightly Coupled Code:

```
// High-level module
class OrderService {
    // Directly depends on low-level MySQL implementation
    private MySQLDatabase database;

    public OrderService() {
        this.database = new MySQLDatabase();  // Concrete dependency
    }

    public void saveOrder(Order order) {
        database.save(order);  // Tight coupling
    }
}

// Low-level module
class MySQLDatabase {
    public void save(Object obj) {
        // MySQL-specific implementation
        String query = "INSERT INTO orders ...";
        // Execute query
    }
}

// Problem: Changing database requires modifying OrderService!
```

DIP Implementation

Dependency Injection + Abstraction:

```java
// Abstraction (Interface)
interface Repository {
    void save(Object obj);
    Object findById(int id);
}

// High-level module
class OrderService {
    // Depends on abstraction, not concrete implementation
    private Repository repository;

    // Dependency injected (Constructor Injection)
    public OrderService(Repository repo) {
        this.repository = repo;
    }

    public void saveOrder(Order order) {
        repository.save(order);  // Uses abstraction
    }
}

// Low-level module depends on abstraction
class MySQLRepository implements Repository {
    public void save(Object obj) {
        // MySQL-specific implementation
    }
    public Object findById(int id) { /* ... */ }
}

class MongoDBRepository implements Repository {
    public void save(Object obj) {
        // MongoDB-specific implementation
    }
    public Object findById(int id) { /* ... */ }
}
```

```java
// Usage - Dependency determined at runtime
Repository repo = new MySQLRepository();  // or MongoDBRepository
OrderService service = new OrderService(repo);
```

DIP Implementation Patterns

Three Dependency Injection Types:

```java
// 1. Constructor Injection (Recommended)
class OrderService {
    private Repository repo;
    public OrderService(Repository repo) {
        this.repo = repo;  // Injected via constructor
    }
}

// 2. Setter Injection
class OrderService {
    private Repository repo;
    public void setRepository(Repository repo) {
        this.repo = repo;  // Injected via setter
    }
}

// 3. Interface Injection
interface RepositoryConsumer {
    void injectRepository(Repository repo);
}
class OrderService implements RepositoryConsumer {
    private Repository repo;
    public void injectRepository(Repository repo) {
        this.repo = repo;
    }
}
```

Dependency Inversion vs Dependency Injection:

- **DIP**: Design principle about abstraction direction
- **DI**: Implementation technique to achieve DIP
- **IoC Containers**: Frameworks that automate DI (Spring, Guice)

When to Apply SOLID Principles

Appropriate Contexts:

- ❑ **Large codebases** with multiple developers
- ❑ **Long-lived projects** requiring maintenance
- ❑ **Frameworks and libraries** used by others
- ❑ **Complex business domains** with changing requirements

When to Be Pragmatic:

- ❑ **Prototypes and MVPs** - focus on speed
- ❑ **Small scripts/tools** - simplicity over structure
- ❑ **Performance-critical sections** - abstractions have cost
- ❑ **Proven stable code** - "if it ain't broke, don't fix it"

Incremental Adoption Strategy:

1. **Start with SRP** - easiest to understand and apply
2. **Add DIP with DI** - enables testability
3. **Apply ISP** when interfaces become bloated
4. **Use OCP** when adding similar features
5. **Ensure LSP** when creating inheritance hierarchies

SOLID & Design Patterns Relationship

Design Pattern	Primary SOLID Principle	Also Supports
Strategy	OCP, DIP	SRP, ISP
Decorator	OCP	SRP
Observer	OCP, DIP	SRP
Factory Method	DIP, OCP	SRP
Adapter	ISP, DIP	SRP
Facade	SRP	DIP
Proxy	DIP, ISP	SRP
Template Method	OCP	SRP, LSP

Patterns as SOLID Solutions:

- Patterns provide **proven implementations** of SOLID principles
- SOLID provides **theoretical foundation** for patterns
- Together they form **robust OOP design methodology**

Applications of SOLID Principles

SOLID principles are applied in:

- ❑ Enterprise applications
- ❑ Microservices architecture
- ❑ API and framework design
- ❑ Scalable backend systems
- ❑ Large codebases

They improve long-term system health.

Summary of the Lecture

In this lecture, we learned:

- Meaning and importance of SOLID principles
- Real-time examples of each principle
- How SOLID improves low-level design
- Why SOLID is essential for scalable systems
- Applying SOLID leads to clean, maintainable code.

Quiz / Check Your Understanding

1. Why is SRP important in large systems?
2. How does OCP prevent regression bugs?
3. What problem does LSP solve?
4. Why is ISP preferred over large interfaces?
5. How does DIP support testability?

Frequently Asked Questions (FAQs)

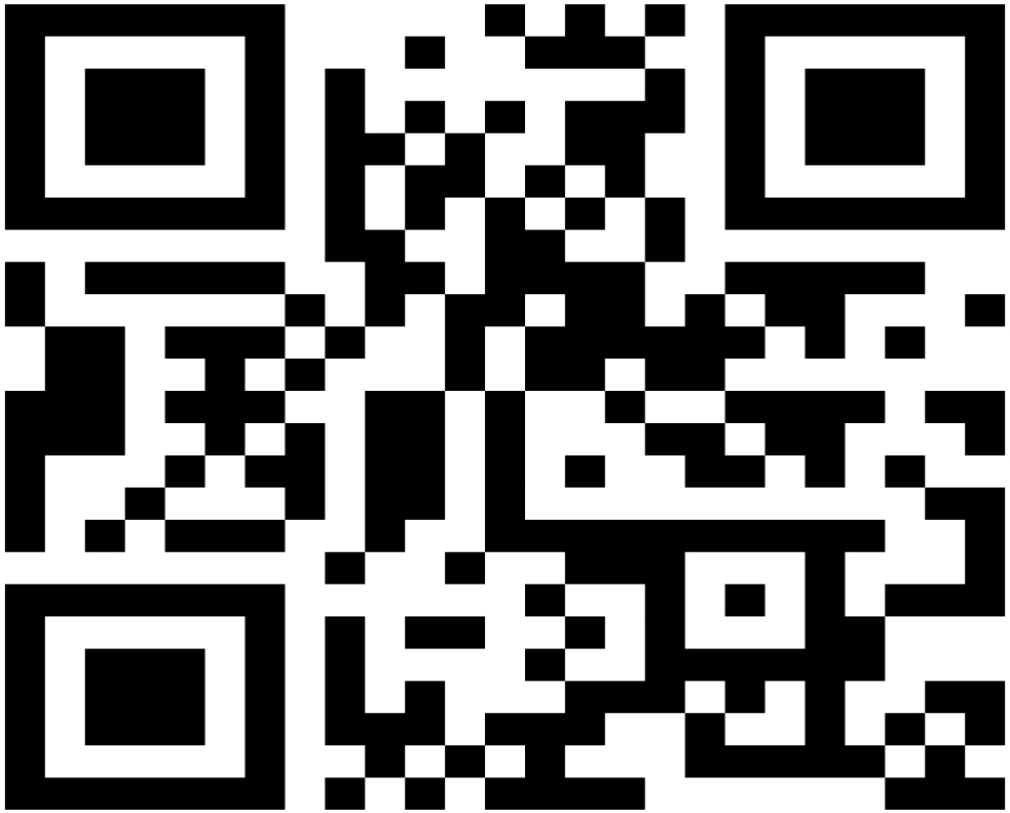
Q: Are SOLID principles mandatory?
A: No, but strongly recommended for maintainable systems.
Q: Can SOLID principles be violated sometimes?
A: Yes, if justified by performance or simplicity needs.
Q: Are SOLID principles used in system design interviews?
A: Yes, frequently.

References / Learning Resources

Faculty-Curated Resources:

NPTEL – *Object-Oriented Analysis and Design*
Coursera – *Object-Oriented Design*
LinkedIn Learning – *SOLID Principles*
Clean Code – Robert C. Martin
Refactoring.Guru – SOLID Principles
GeeksforGeeks – SOLID Tutorials

Class-Wise Feedback



Thank You

For queries
Email: mayank.e18387@cumail.in